

Computación Paralela y Distribuida

2026-II

José Fiestas PhD

25 de agosto de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Objetivos:

2.1 Velocidad, eficiencia, escalabilidad. Ley de Ahmdal

2.2 Speedup, performance

2.3 **Modelos computacionales en paralelo - PRAM (1)**

2.4 Modelos computacionales en paralelo - PRAM (2)

2.5 Particionamiento de datos y tareas

2.6 Random en paralelo

2.7 N-body, Divide y Vencerás (mergesort)

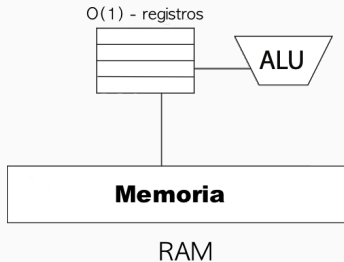
Al finalizar la sesión, los alumnos conocen:

- Modelo y clasificación PRAM (Parallel Random Access Machine)
- Teorema de Brent
- Pseudocódigo (modelo formal) PRAM
- Extensiones del modelo PRAM

U2.3 Modelos computacionales en paralelo (PRAM)

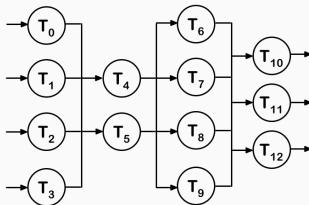
Modelo RAM (Random Access Machine)

El modelo **RAM** (modelo Von Neumann) es una abstracción de un computador secuencial con memoria infinita, en el cual cada espacio de memoria puede ser accedida en forma directa. Contiene instrucciones para lectura, escritura, y operaciones aritméticas/lógicas. Es útil para el análisis de algoritmos secuenciales.



Modelos Computacionales en Paralelo

Un **modelo computacional** en paralelo es una abstracción de la funcionalidad de un sistema de acceso, manejo y almacenamiento de información, que es independiente del hardware/software específico que se utiliza.

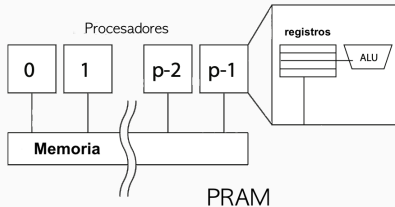


Define cómo se organizan los cálculos y la comunicación cuando **múltiples procesadores trabajan simultáneamente** para resolver un problema.

Modelo PRAM (Parallel Random Access Machine)

El modelo **PRAM** consiste en un conjunto de *procesadores* idénticos $\{P_1, P_2, \dots, P_n\}$, con acceso a una memoria compartida para lectura/escritura de datos. Cada *procesador* es un RAM, que ejecuta el mismo código en forma sincrónica. No existe una red entre *procesadores*, si no que se comunican a través de la memoria común. Cada *procesador*, en cada paso, ejecuta:

- lectura de data de la memoria común
- ejecución local de instrucciones
- escritura de data en la memoria común



Pseudocódigo por *procesador*

E.g.

Sea una instrucción de la forma $c := a + b$

Cada *procesador* ejecuta:

Algorithm 1: Suma con lectura y escritura global

Input : Variables globales a, b

Output: c tal que $c = a + b$

```
1 global_read  $a, x$  ;           // copiar  $a$  en la memoria local  $x$ 
2 global_read  $b, y$  ;           // copiar  $b$  en la memoria local  $y$ 
3  $z \leftarrow x + y$ ;
4 global_write  $z, c$  ;           // escribir  $z$  en el espacio  $c$ 
```

donde $a, b, y c$ son espacios de memoria compartidos.

Clasificación PRAM

PRAM: clasificación

Clasificación según acceso en paralelo (en el mismo paso) a direcciones de memoria: {exclusive, concurrent} **read** {exclusive, concurrent} **write**

- **EREW**, cada locación de memoria puede ser leída o escrita por un solo *procesador* a la vez
- **CREW**, múltiples *procesadores* leen el mismo espacio de memoria pero la escritura es exclusiva en memoria
- **ERCW**, se permite escritura concurrente, pero la lectura es exclusiva por cada proceso (en el mismo paso). No es muy usada
- **CRCW**, múltiples *procesadores* leen y escriben en la misma dirección de memoria

EREW → CREW → CRCW

E.g. búsqueda de un valor en un array de dimensión n ¿Cuál será la clasificación PRAM?

Teorema de Brent

Tiempo de ejecución en paralelo: número de pasos que ejecuta un algoritmo usando p *procesadores*

$$T_p(n)$$

Costo: cantidad total de recursos computacionales consumidos durante la ejecución.

$$C_p(n) = T_p(n)P(n)$$

El **Span** (T_∞), tiempo del algoritmo suponiendo un número ilimitado de *procesadores* (longitud del camino crítico)

Trabajo: cantidad total de operaciones realizada por el algoritmo.

$$W(n)$$

Un algoritmo es de **costo óptimo** si

$$C_p(n) = T_p(n)P(n) = \Theta(T_s(n))$$

Paradigma WT (work-time)

Principio WT de asignación de tareas:

Para el cálculo de $T_p(n, p)$:

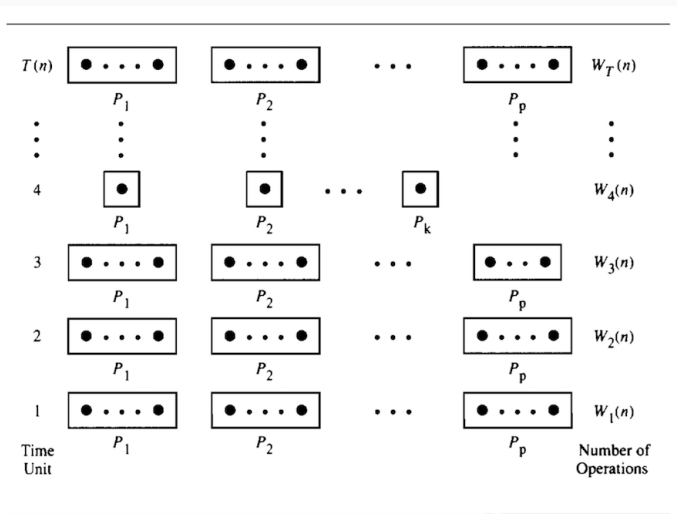
Límite inferior: cantidad de operaciones concurrentes por paso $\geq \frac{W(n)}{p}$

Límite superior: dados p procesadores, y un algoritmo con span $T_\infty(n)$, y trabajo $W(n)$ (cantidad total de operaciones), un algoritmo PRAM puede ejecutarse en $\leq \lfloor \frac{W(n)}{p} \rfloor + T_\infty(n)$

Demostración segun [1]:

Sea $W_i(n)$ la cantidad de operaciones ejecutadas en el paso i , donde $1 \leq i \leq T_\infty(n)$. Si se ejecuta cada $W_i(n)$ operaciones en $\leq \lceil \frac{W_i(n)}{p} \rceil$ pasos en paralelo por los p procesadores. Entonces, el algoritmo PRAM logrará ejecutarse en $T(n, p) = \sum_i \lceil \frac{W_i(n)}{p} \rceil \leq \sum_i (\lfloor \frac{W_i(n)}{p} \rfloor + 1) = \lfloor \frac{W(n)}{p} \rfloor + T_\infty(n)$ pasos en paralelo

Paradigma WT (work-time)



Paradigma WT (work-time scheduling principle [1] Fig.1.9 p.29)

Dado un algoritmo paralelo con span $T_{\infty}(n)$ y un total de $W(n)$ operaciones (tareas), este puede simularse en un PRAM de p procesadores en

$$T_p(n, p) = O\left(\frac{W(n)}{p} + T_{\infty}(n)\right)$$

El costo (máximo) está definido como

$$C_p(n, p) = T_p(n, p) \cdot p = O(W(n) + T_{\infty}(n) \cdot p)$$

$W(n)$ y $C_p(n)$ coinciden cuando $p = O\left(\frac{W(n)}{T_{\infty}(n)}\right)$

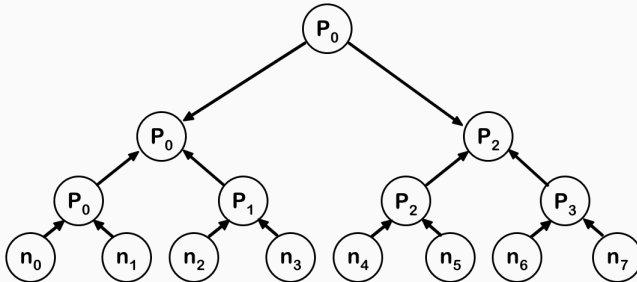
Brent (suma de números)

$$W(n)=O(n) , T_{\infty}(n) = O(\log n)$$

$$T_p(n, p) = O\left(\frac{n}{p} + \log n\right)$$

$$C_p(n, p) = O(n + p \log n)$$

$$\text{Si } p = O\left(\frac{W(n)}{T_{\infty}(n)}\right) = O\left(\frac{n}{\log n}\right), C_p(n) = O\left(n + \frac{n}{\log n} \log n\right) = O(n) = W(n)$$



Modelos formales PRAM

Pseudocódigo (secuencial)

```
Input: an RE  $r$  and a string  $T = a_1 \cdots a_n$ , where  $a_i \in \Sigma$ .  
Output:  $\text{Match}(r, T)$ .  
1: Generate a T-NFA  $M = (Q, \Sigma, \delta, q_0, q_f)$  from  $r$ .  
2: Sort  $Q$  in topological order.  
3: for all  $q \in Q$  do  
4:    $\text{Start}[q] \leftarrow 0$   
5: end for  
6: Compute  $Q_f = \{q \mid q \text{ is an } s\text{-state such that there is an } \varepsilon\text{-path from } q \text{ to } q_f\}$   
7: for  $i = 1$  to  $n$  do  
8:    $\text{UpdateStart}(M, \text{Start}, i)$   
9:    $\text{pos} = \max_{q \in Q_f} \text{Start}[q]$   
10:  if  $\text{pos} > 0$  then  
11:     $\text{Output}(\text{pos}, i)$   
12:    for all  $q \in Q$  do  
13:      if  $\text{Start}[q] \leq \text{pos}$  then  
14:         $\text{Start}[q] \leftarrow 0$   
15:      end if  
16:    end for  
17:  end if  
18: end for
```

Ejemplo de pseudocódigo. De: Yamamoto, 2018, *A faster algorithm for finding shortest substring matches of a regular expression*

El lenguaje formal utiliza **pardo** (do in parallel).

Condición: **n=p**

Algorithm 2: Se asignan n elementos de A a B en paralelo

```
1 for  $1 \leq i \leq n$  pardo  
2    $B(i) \leftarrow A(i);$ 
```

Es decir, si $n=p$, un elemento por *procesador*.

El bloque **for** se ejecuta en forma **concurrente**.

I.e. *procesador* P1 asigna B(1) a A(1), *procesador* P2 asigna B(2) a A(2),
etc

PRAM (Parallel RAM)

Algorithm 1 Parallel Regular Expression Matching (PaREM)

```
%Input: Transition table Tt, set of final states F, input T%
%Output: Result of REM%
1:  $I = \text{vector}(p)$  /* initialize final result vector */
   % $P_0 \dots P_p \dots$  processing unit,  $p$  is the total number of
   processing units %
2: for  $P_0, P_1, \dots, P_p$  do in parallel
3:    $\text{start\_position} = i * (T.\text{length}/p)$ 
4:    $\text{pi\_input} = \text{substring}(\text{start\_position}, T.\text{length}/p)$ 
   %start find possible initial states %
5:   for  $q_0, q_1, \dots, q_n$  do
   %  $\text{pi\_input.at}(0)$  returns the first char of  $\text{pi\_input}$  %
6:     if  $(Tt[q_i][\text{pi\_input.at}(0)] \in Q)$  then
7:        $S[i] = q_i$ 
8:     end if
9:   end for
10:  for  $q_0, q_1, \dots, q_n$  do
   %  $\text{pi\_input.back}()$  returns the last char of  $\text{pi\_input}$  %
11:    if  $(Tt[q_i][\text{pi\_input.back}()] \in Q)$  then
12:       $L[i] = Tt[q_i][\text{pi\_input.back}()]$ 
13:    end if
14:  end for
```

Ejemplo de pseudocódigo en paralelo. De: *Memeti & Pllana, 2015, PaREM: A Novel Approach for Parallel Regular Expression Matching*

Algorithm 3: Parallel Global OR

```
1 Input: array  $x[1, \dots, n]$  ;  
2 Output: Result;  
3 Result  $\leftarrow 0$  ;  
4 for  $i = 1, \dots, n$  pardo  
5   |   if  $x[i]$  then  
6   |   |   Result  $\leftarrow 1$  ;
```

Cada *procesador* evalúa la condición **if** y modifica el valor de Result si $x[i] = 1$

Algorithm 4: Parallel Global AND

```
1 Input: array  $x[1, \dots, n]$  ;  
2 Output: Result;  
3 Resultado  $\leftarrow 1$  ;  
4 for  $i = 1, \dots, n$  pardo  
5   | if not  $x[i]$  then  
6   |   | Result  $\leftarrow 0$ ;
```

Cada *procesador* evalúa la condición **if** y modifica el valor de Result si $x[i] = 0$

PRAM Caso 2: Máximo de n números (CRCW)

Algorithm 5: Cálculo del máximo de n números

```
1 Input: array  $A[1, \dots, n]$  ;  
2 Output: array  $M[1, \dots, n]$             $M[i] = 1$  iff  $A[i] = \max_j A[j]$  ;  
3 forall  $(i, j) \in \{1, \dots, n\}^2$  pardo  
4    $B[i, j] \leftarrow A[i] \geq A[j]$  ;  
5 forall  $i \in \{1, \dots, n\}$  pardo  
6    $M[i] \leftarrow \bigwedge_{j=1}^n B[i, j]$  ;
```

1. Cada *procesador* compara el valor de un elemento del array con otro del mismo array A
2. El índice del elemento de A donde el Global AND del array booleano M sea 1 será el máximo de A

PRAM Caso 3: Máximo de n números (CRCW)

Secuencial

$$P(n)=1, T_s(n) = O(n)$$

Paralelo:

El máximo de n números se puede calcular en $O(1)$ usando n^2 procesadores, lo que lo hace impracticable para n grande.

$$P(n)=O(n^2), T_\infty(n) = O(1) \rightarrow C_p(n)=O(n^2)$$

$$S(n)=O(n)/O(1)=O(n)$$

$$E(n)=O(n)/O(n^2)=O(1/n)$$

Optimiza $T(n)$, sin considerar $P(n)$.

Alto speedup, baja eficiencia

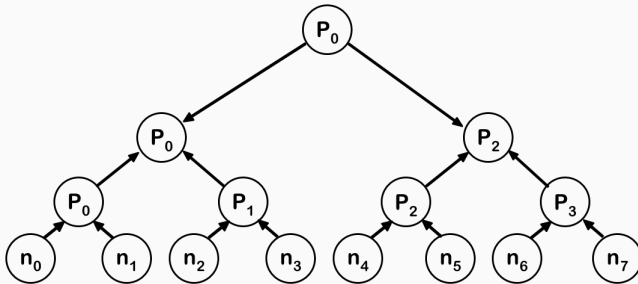
PRAM Caso 4: Suma de n números

Algorithm 6: Suma de n números en paralelo

```
1 Input: vector  $A$  con  $n = 2^k$  elementos ;
2 Output: suma  $S$  de elementos de  $A$  ;
3 for  $i = 1$  to  $n$  pardo
4    $B[i] \leftarrow A[i]$  ;                                // paso 1
5 for  $h = 1$  to  $\log(n)$  do
6   for  $i = 1$  to  $n/2^h$  pardo
7      $B[i] \leftarrow B[2i - 1] + B[2i]$  ;              // paso 2
8  $S \leftarrow B[1]$  ;                                  // paso 3
```

PRAM: Suma de n números

Para $n=8$, $p=8$



Secuencial

$$P(n)=1, T_s(n) = O(n)$$

Paralelo:

En el paso 1 se ejecutan n operaciones

En el paso 2 se ejecutan $\sum_{j=1}^{\log(n)} n/2^j$

En el paso 3 se ejecuta 1 operación

Por lo tanto:

$$W(n) = W_1(n) + W_2(n) + W_3(n) = n + \sum_{j=1}^{\log(n)} n/2^j + 1 = O(n)$$

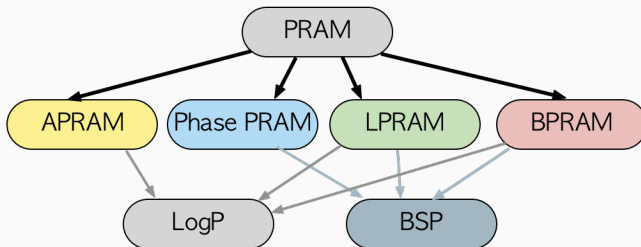
$$T_{\infty}(n) = O(\log(n) + 2) = O(\log(n))$$

Este algoritmo ejecuta un modelo PRAM CREW

Extensiones del modelo PRAM

Extensiones del modelo PRAM

Los modelos PRAM clásicos (EREW, CREW, CRCW) solo distinguen lectura/escritura concurrente. Los modelos extendidos de PRAM son:



APRAM	:	asincronía
Phase PRAM	:	sincronización
LPRAM	:	localidad
BPRAM	:	volumen y granularidad de comunicación
BSP	:	cómputo + comunicación + sincronización
LogP	:	asincronía + latencia + overhead + ancho de banda

Modelo BSP (bulk synchronously parallel)

Es una computadora que consiste en *procesadores* con memoria local y capaz de comunicarse punto a punto a través de una red.

p *procesadores*

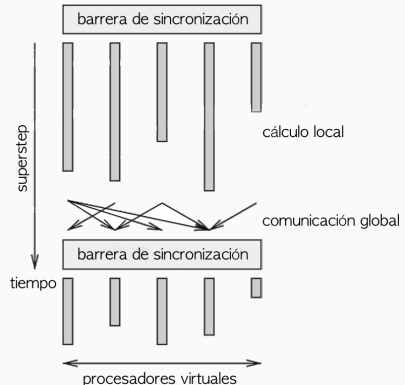
virtuales, que ejecutan
cálculos en cada superstep

El tiempo
de ejecución de un superstep
($T_{superstep}$) esta dado por:

$$T_{superstep} = \max(w_i) + h \cdot g + L$$

Donde w_i es el tiempo
necesitado por *procesador* i , h
es el número máximo de palabras
enviadas/recibidas por *procesador*,
 g es el costo de comunicación por
palabra (bandwidth parameter),

$h \cdot g$ es el tiempo total de comunicación y L es el tiempo de la barrera de sincronización.



Modelo BSP (bulk synchronously parallel)

Algorithm 7: Reducción paralela de suma

Input: Arreglo $A[0, \dots, n-1]$

Output: $A[0] = \sum_{i=0}^{n-1} A[i]$

```
1  $step \leftarrow 1$ 
2 while  $step < n$  do
3   for  $i = 0$  to  $\lfloor n/(2\ step) \rfloor - 1$  pardo
4      $A[2\ step\ i] \leftarrow A[2\ step\ i] + A[2\ step\ i + step]$ 
5   BSP_SYNC
6    $step \leftarrow 2\ step$ 
```

Modelo BSP (bulk synchronously parallel)

Complejidad:

Superpaso : $\log p$ superpasos

En cada uno:

cada procesador envía 1 dato

cada procesador recibe 1 dato

Entonces:

$w=1$, $h=1$

tiempo por superpaso: $T_{sp} = 1 + g + L$

Tiempo total de reducción: $T_{red} = \log p(1 + g + L)$

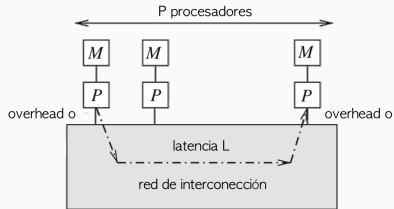
$$T_{BSP} = O\left(\frac{n}{p} + g \log p + L \log p\right)$$

Modelo log P

Modelo log P

Tiene la estructura de BSP, i.e. un sistema de memoria distribuido, con los siguientes parámetros:

- **L** (latencia), es un límite superior de la latencia de la red, al enviar un mensaje pequeño.
- **o** (overhead), el tiempo requerido en comunicación, durante el cual no se puede ejecutar ninguna otra operación
- **g** (gap), es el tiempo mínimo entre envío/recibo de mensajes consecutivos por *procesador*
- **P** (*procesadores*), el número de procesadores de la máquina



Algorithm 8: Reducción paralela de suma para el modelo LogP

Input: Arreglo $A[0, \dots, n-1]$

Output: $A[0] = \sum_{i=0}^{n-1} A[i]$

```
1  $d \leftarrow 1$ 
2 while  $d < n$  do
3   for  $i = 0$  to  $\lfloor n/(2d) \rfloor - 1$  pardo
4      $x \leftarrow A[2di + d]$ 
5     send( $x, 2di$ )
6      $y \leftarrow \text{receive}()$ 
7      $A[2di] \leftarrow A[2di] + y$ 
8    $d \leftarrow 2d$ 
```

Complejidad

Reducción log p

While indica log p iteraciones. En cada iteración cada *procesador* activo envía y recibe 1 mensaje. Además, realiza 1 suma.

El costo de una iteración es aproximadamente $T = L + o + g$

Total reducción:

$$T_{red} = \log p (L + o + g)$$

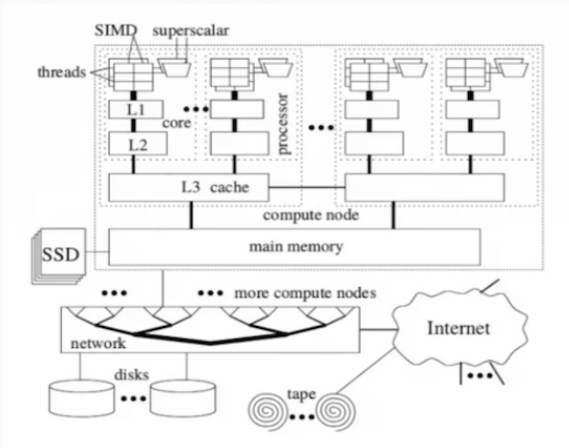
$$T_{\log P} = O \left(\frac{n}{p} + \log p (L + o + g) \right)$$

donde $\frac{n}{p}$ es el cómputo local, $o \log p$ es el overhead CPU, $L \log p$ es la latencia, y $g \log p$ es el ancho de banda

LogP separa el costo de comunicación con mayor detalle.

Niveles de Paralelismo

Estrategia es crear modelos con pocos niveles de paralelismo (híbridos)



DAG → PRAM:

Cada nivel del DAG que puede ser paralelizado, se representa como un modelo PRAM

PRAM → DAG:

Identificar bucles y condicionales con distintos *procesadores* en PRAM

DAG → Red:

- Nodos son *procesadores*
- Aristas son líneas de comunicación en la red
- Tiempo de ejecución depende de los nodos, aristas, y de la longitud de las rutas en cada nivel del DAG.



Joseph Jája *An introduction of parallel algorithms*. University of Maryland Ch1.3.2., 1.5.

